T1 染色条（stripe）

固定最后留下的黑色区间

最后只要有一段连续 k 格全部变黑，目标就能达到。因此，可以枚举这段区间的位置。区间中的白格都必须染色，黑格不必改动，区间外也不必染色，所以这段区间的最少代价恰好是其中的白格数。对所有长度为 k 的区间取最小值，就不会遗漏最优答案。

相邻两个区间只相差一进一出的两个格子，可以用滑动窗口维护白格数 white：右端加入一个白格就加一；窗口超过 k 格时，移走最左格，若它是白格就减一。每次得到恰好 k 格的窗口，更新最小值。

代码中的字符串下标从 0 开始。处理位置 i 后，先移除可能多出的 $i-k$，再在 $i \geq k-1$ 时更新答案，因此首个窗口、最后一个窗口和 $k=1$ 都被包含。初始化答案为 k，对应把某个窗口全部染黑的可行上界。

复杂度

单组时间 $O(n)$，保存字符串占空间 $O(n)$。全部测试组的总时间为 $O(\sum n)$，最多处理 2×10^5 个格子。

参考代码

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    freopen("stripe.in", "r", stdin);
    freopen("stripe.out", "w", stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t;
    cin >> t;
    while (t--) {
        int n, k;
        string s;
        cin >> n >> k >> s;
        int white = 0, ans = k;
        for (int i = 0; i < n; i++) {
            if (s[i] == 'W') white++;
            if (i >= k && s[i - k] == 'W') white--;
            if (i >= k - 1) ans = min(ans, white);
        }
        cout << ans << '\n';
    }
    return 0;
}
```

部分分

下表分值为算法保证分；测试点按点独立计分。

条件	保证分	做法与复杂度
各组 n 总和不超过 2000（点 $1 \sim 6$）	30 分	枚举长度 k 区间的左端点，逐格数其中白格，取最小值。单组时间 $O((n-k+1)k)$，不超过 $O(nk)$。

源码为 部分分代码/stripe_1.cpp。当某个窗口的白格数已经为 0，可以立即结束本组，因为答案不可能更小。

T2 整倍调整（multiple）

哪些时刻能够调整同一个数

一个位置至多被修改一次，因此，原来不能整除的 a_i 必须在某个满足 $a_i + x \equiv 0 \pmod{k}$ 的时刻被选择。记它还需要的余数为

$$r_i = (k - a_i \bmod k) \bmod k.$$

如果 $r_i = 0$，这个数已经满足要求，直接不选它即可。否则，可以选择它的计数器值依次是 $r_i, r_i + k, r_i + 2k, \ldots$。

问题中的冲突来自“一步只能选一个位置”：需要同一个非零余数 r 的数，不能都用时刻 r。若这样的数有 c 个，最后一个至少要等到 $r + (c-1)k$，而把它们依次放到前 c 个可用时刻就能达到这个下界。

不同余数的可用时刻不会相同，因为一个整数除以 k 只有一个余数。因此各类的安排可以同时执行，不会出现新的冲突。所有类中最晚的调整时刻，就是完成全部调整所需的最晚时刻。

要注意，开始时 $x = 0$。在 $x = L$ 时完成最后一次调整后，总共进行的是 $L + 1$ 步，而不是 L 步。于是，每个非空余数类给出的候选答案为

$$r + (c-1)k + 1.$$

把所有非零需求余数排序，相同余数会连续出现；扫描每个连续段，求出其数量并更新最大值。若没有非零余数，直接输出 0。

复杂度与整数范围

单组排序时间 $O(n \log n)$，其余处理 $O(n)$，数组空间 $O(n)$。全部测试组总时间为 $O(\sum n \log n)$，不超过 $O((\sum n) \log(2 \times 10^5))$。

答案不超过 $nk \le 2 \times 10^{14}$，计算次数与 k 的乘积必须使用 long long。不能只把最终答案声明为 64 位，而让中间乘法先在 32 位中溢出。

参考代码

```cpp
#include <bits/stdc++.h>
using namespace std;
long long need[200005];

int main() {
    freopen("multiple.in", "r", stdin);
    freopen("multiple.out", "w", stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t;
    cin >> t;
    while (t--) {
        int n;
        long long k, x;
        cin >> n >> k;
        int cnt = 0;
        for (int i = 1; i <= n; i++) {
            cin >> x;
            long long r = (k - x % k) % k;
            if (r != 0) need[++cnt] = r;
        }
        sort(need + 1, need + cnt + 1);
        long long ans = 0;
        for (int l = 1; l <= cnt; ) {
            int r = l;
            while (r <= cnt && need[r] == need[l]) r++;
            long long times = r - l;
            ans = max(ans, need[l] + (times - 1) * k + 1);
            l = r;
        }
        cout << ans << '\n';
    }
    return 0;
}
```

部分分

下表分值为累计保证分，第二行已包含第一行，不重复相加。

条件	保证分	做法与复杂度
$k \le 30$ 且各组 n 总和不超过 100（点 $1 \sim 5$）	25 分	从 $x = 0$ 逐步模拟，每步扫描所有未完成位置，若存在加 x 后可整除者，就处理其中一个，否则空过。至多 nk 步，单组时间 $O(n^2k)$。
各组 n 总和不超过 2000（点 $1 \sim 10$）	50 分	算出每个非零需求余数；对每个位置扫描全数组，统计同类数量，再使用正文的末时刻公式取最大值。单组时间 $O(n^2)$。

源码依次为 部分分代码/multiple_1.cpp、部分分代码/multiple_2.cpp。模拟时，同一时刻能被处理的位置属于同一余数类，选哪一个都不会影响后续可用时刻；本来可整除的位置一开始就标记完成。

T3 花束（flowers）

从最后一段花接出递推

要数出长度为 i 的合法序列，可以按末尾的颜色拆开：如果最后是红花，删掉这一朵，剩下的仍是合法序列；如果最后是白花，最后一段白花至少有 k 朵，删掉最后 k 朵后，剩下的仍然合法。

反过来，在合法序列末尾接一朵红花一定合法；接上 k 朵白花也一定合法——原来以白花结尾时，末段长度增加 k，仍是 k 的倍数。因此，这两种拆法分别与长度 $i-1$、$i-k$ 的合法序列一一对应，且末尾颜色不同，不会重复计数。

能否这样接花只依赖剩余序列是否合法、长度是多少，不需要再记录它末尾有几朵白花。于是定义 f_i 为恰好排 i 朵花的合法序列数。

$$f_0 = 1, \qquad f_i = \begin{cases} f_{i-1}, & i < k, \\ f_{i-1} + f_{i-k}, & i \geq k. \end{cases}$$

$f_0 = 1$ 表示唯一的空序列，它用来接出第一朵红花或第一组白花；题目询问的长度至少为 1，不会把空序列作为答案计入。按 i 递增计算时，右边依赖的状态都已经得到。每次相加后取模。

把多个长度的答案汇总

一次询问要求的是 $f_a + f_{a+1} + \cdots + f_b$。所有询问使用同一个 k，所以 f 只需算一次；再维护

$$\mathrm{pre}_0 = 0, \qquad \mathrm{pre}_i = (\mathrm{pre}_{i-1} + f_i) \bmod (10^9 + 7).$$

答案就是 $(\mathrm{pre}_b - \mathrm{pre}_{a-1} + 10^9 + 7) \bmod (10^9 + 7)$。减去的是 $a-1$ 而不是 a；加上模数是为了避免 C++ 对负数取余得到负结果。

复杂度

代码预处理到完整长度上限 $M = 10^5$。时间为 $O(M + t)$，每次询问只做常数次运算，空间为 $O(M)$。

参考代码

```cpp
#include <bits/stdc++.h>
using namespace std;
const int MOD = 1000000007;
int f[100005], pre[100005];

int main() {
    freopen("flowers.in", "r", stdin);
    freopen("flowers.out", "w", stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t, k;
    cin >> t >> k;
    f[0] = 1;
    for (int i = 1; i <= 100000; i++) {
        f[i] = f[i - 1];
        if (i >= k) f[i] = (f[i] + f[i - k]) % MOD;
        pre[i] = (pre[i - 1] + f[i]) % MOD;
    }
    while (t--) {
        int a, b;
        cin >> a >> b;
        cout << (pre[b] - pre[a - 1] + MOD) % MOD << '\n';
    }
    return 0;
}
```

部分分

记 B 为所有询问的最大右端点，$M = 10^5$。下表为累计保证分，三行不相加。

条件	保证分	做法与复杂度
$t \le 10$ 且 $B \le 16$（点 $1 \sim 4$）	20 分	对每个长度枚举红白序列，遇到红花或序列结束时检查刚结束的白花段长度是否为 k 的倍数；统计每个长度后逐问累加。时间上界 $O(2^B + tB)$。
$t \le 500$ 且 $B \le 500$（点 $1 \sim 8$）	40 分	每次询问分别计算 f_0 到 f_b，把下标位于 $[a, b]$ 的值相加。时间 $O(\sum b)$，不超过 $O(tB)$。
前两档，或每次 $b - a \le 9$（点 $1 \sim 12$）	60 分	共用一次长度递推，但不建立前缀和；每次询问逐项累加 f_a 到 f_b。时间 $O(M + \sum(b - a + 1))$，窄区间档每次至多累加 10 项。

源码依次为 部分分代码/flowers_1.cpp、部分分代码/flowers_2.cpp、部分分代码/flowers_3.cpp。白花段结束时才检查长度，不能把中间暂时不足 k 的白花前缀直接判成无解。

T4 喂猫（cats）

最后一次投喂限制了此前的时间

按时间从左到右考虑。若在时刻 i 投喂，那么所有此时在场的猫都会吃到。设它们有 c_i 只；当 $c_i > 0$ 时，再记这些猫最早的到场时刻为 p_i，即它们的最小左端点。

为什么只需这个最小左端点？到场时间为 p_i 的那只猫一直在场到时刻 i。此前如果在 $[p_i, i-1]$ 中任意时刻投喂过，它就会吃到第二次。因此，此前的所有投喂必须早于 p_i。

这个限制也足够：当前在场的每只猫都不早于 p_i 到场，所以在 $p_i - 1$ 及以前投喂过的猫，与本次投喂的猫不会重复。于是，固定最后一次投喂在 i 后，前半段只需知道“到 $p_i - 1$ 为止最多能喂多少只”，不必记具体喂过哪几只。

据此定义 f_i 为只在前 i 个时刻安排投喂时，最多能喂到的猫数。初始 $f_0 = 0$，表示不投喂。

- 不在 i 投喂，答案来自 f_{i-1}。
- 在 i 投喂，先取前 $p_i - 1$ 个时刻的最优安排，再新增当前的 c_i 只猫，得到 $f_{p_i-1} + c_i$。

所以，当 $c_i > 0$ 时，

$$f_i = \max\bigl(f_{i-1}, f_{p_i-1} + c_i\bigr).$$

当 $c_i = 0$ 时，投喂没有收益，直接令 $f_i = f_{i-1}$。这两种选择覆盖所有方案，并且第二种拼接不会重复喂猫；从小到大计算即可得到答案 f_n。

维护在场数量和最早到场时间

每只猫在整个闭区间 $[l, r]$ 中贡献 1。使用差分数组，在 diff[l] 加一，在 diff[r+1] 减一，扫描时累加就得到当前在场数 count。同一区间的猫也要分别累加，不能去重。

要维护最小左端点，对每个到场时刻 l，记录从这一时刻到场的猫中最晚离开的时间 last[l]；没有猫从这里到场时为 0。当 $l \le i$ 时，从 l 到场的猫仍有在场者，当且仅当 last[l] >= i。

用指针 p 从 1 向右移动，跳过所有 last[p] < i 的左端点。这些猫在当前时刻已经全部离开，以后也不会重新出现，所以被跳过的位置永远不用回头检查。停止时，如果有猫在场，p 就是所需的 p_i；若无人，指针可以到 $i + 1$，这时不进行投喂转移。

多组数据之间，把本组会访问的差分、最晚离开时间和 DP 数组清零。差分会写到 $n + 1$，清空和数组容量都要包括这个位置。

复杂度

读入每只猫只做常数次修改，每个时刻处理一次，指针最多前进 n 次。单组时间 $O(n + m)$，空间 $O(n)$；全部测试组总时间 $O(\sum n + \sum m)$，不超过约 1.2×10^6 次线性处理单位。答案至多为 m，使用 int 足够。

参考代码

```cpp
#include <bits/stdc++.h>
using namespace std;
int diff[1000005], last[1000005], f[1000005];

int main() {
    freopen("cats.in", "r", stdin);
    freopen("cats.out", "w", stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t;
    cin >> t;
    while (t--) {
        int n, m;
        cin >> n >> m;
        for (int i = 0; i <= n + 1; i++) {
            f[i] = 0;
            last[i] = 0;
            diff[i] = 0;
        }
        for (int i = 1; i <= m; i++) {
            int l, r;
            cin >> l >> r;
            diff[l]++;
            diff[r + 1]--;
            last[l] = max(last[l], r);
        }
        int p = 1, count = 0;
        for (int i = 1; i <= n; i++) {
            count += diff[i];
            while (p <= i && last[p] < i) p++;
            f[i] = f[i - 1];
            if (count > 0) f[i] = max(f[i], f[p - 1] + count);
        }
        cout << f[n] << '\n';
    }
    return 0;
}
```

部分分

下表为累计保证分，三行不相加；多组数据的时间成本按各组相加。

条件	保证分	做法与复杂度
各组 n 总和不超过 14，各组 m 总和不超过 20（点 $1\sim4$）	20 分	枚举每个时刻是否投喂，建立所选时刻的前缀计数。检查每只猫的区间内是否至多选中一次，合法时统计被喂猫数并取最大。单组时间 $O(2^n m)$。
各组 n、m 总和分别不超过 2000（点 $1\sim8$）	40 分	对每个时刻扫描全部猫，得到在场数和最小左端点，再执行正文的 DP。单组时间 $O(nm)$。
前两档，或每只猫的区间长度至多为 5（点 $1\sim12$）	60 分	逐只猫遍历整个在场区间，给沿途在场数加一，并更新最小左端点，最后执行 DP。时间 $O(n+\sum(r_i-l_i+1))$；短区间档为 $O(n+5m)$。

源码依次为 部分分代码/cats_1.cpp、部分分代码/cats_2.cpp、部分分代码/cats_3.cpp。不能把一只猫对应为“选一次投喂”，因为一次投喂会同时影响所有在场猫；枚举的对象必须是时刻集合。